Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра интеллектуальных информационных технологий

Лабораторная работа № 7
По дисциплине: «Современные платформы программирования»
За 6 семестр

Выполнил:
Студент 3 курса
Группы ПО-12
Гомберг Д.П.
Проверил:
Кулик А.Д.

Брест 2026

Цель: освоить возможности языка программирования Python в разработке оконных приложений

Вариант – 3

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

• Реализовать соответствующие классы, указанные в задании;

• Организовать ввод параметров для создания объектов (использовать экранные компоненты);

• Осуществить визуализацию графических примитивов

3) Изобразить четырехугольник, вращающийся в плоскости формы вокруг своего центра тяжести.

Задание 2. Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

3) Треугольная салфетка Серпинского

Код программы:

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import math


class RotatingQuadrilateral:
 def __init__(self, canvas, center_x, center_y, width, height, angle=0):
 self.canvas = canvas
 self.center_x = center_x
 self.center_y = center_y
 self.width = width
 self.height = height
 self.angle = angle
 self.polygon_id = None
 self.color = "#3498db"

 def get_vertices(self):
 hw = self.width / 2
 hh = self.height / 2
```

```
        vertices = [
            (-hw, -hh),
            (hw, -hh),
            (hw, hh),
            (-hw, hh)
        ]
        rad = math.radians(self.angle)
        cos_a = math.cos(rad)
        sin_a = math.sin(rad)
        rotated = []
        for x, y in vertices:
            new_x = self.center_x + x * cos_a - y * sin_a
            new_y = self.center_y + x * sin_a + y * cos_a
            rotated.append((new_x, new_y))
        return rotated

    def draw(self):
        if self.polygon_id:
            self.canvas.delete(self.polygon_id)
        vertices = self.get_vertices()
        flat_vertices = [coord for vertex in vertices for coord in vertex]
        self.polygon_id = self.canvas.create_polygon(
            flat_vertices, fill=self.color, outline="black", width=2
        )

    def update_angle(self, delta_angle):
        self.angle = (self.angle + delta_angle) % 360
        self.draw()


class SierpinskiTriangle:
    def __init__(self, canvas, p1, p2, p3, depth, color_map=None):
        self.canvas = canvas
        self.p1 = p1
        self.p2 = p2
        self.p3 = p3
        self.depth = depth
        self.color_map = color_map or self._default_colors()
        self.triangle_ids = []

    def _default_colors(self):
        return ["#2ecc71", "#27ae60", "#1e8449", "#145a32", "#0b5345", "#082032"]

    def draw(self):
        for tid in self.triangle_ids:
            self.canvas.delete(tid)
        self.triangle_ids = []
        self._draw_triangle(self.p1, self.p2, self.p3, self.depth)
```

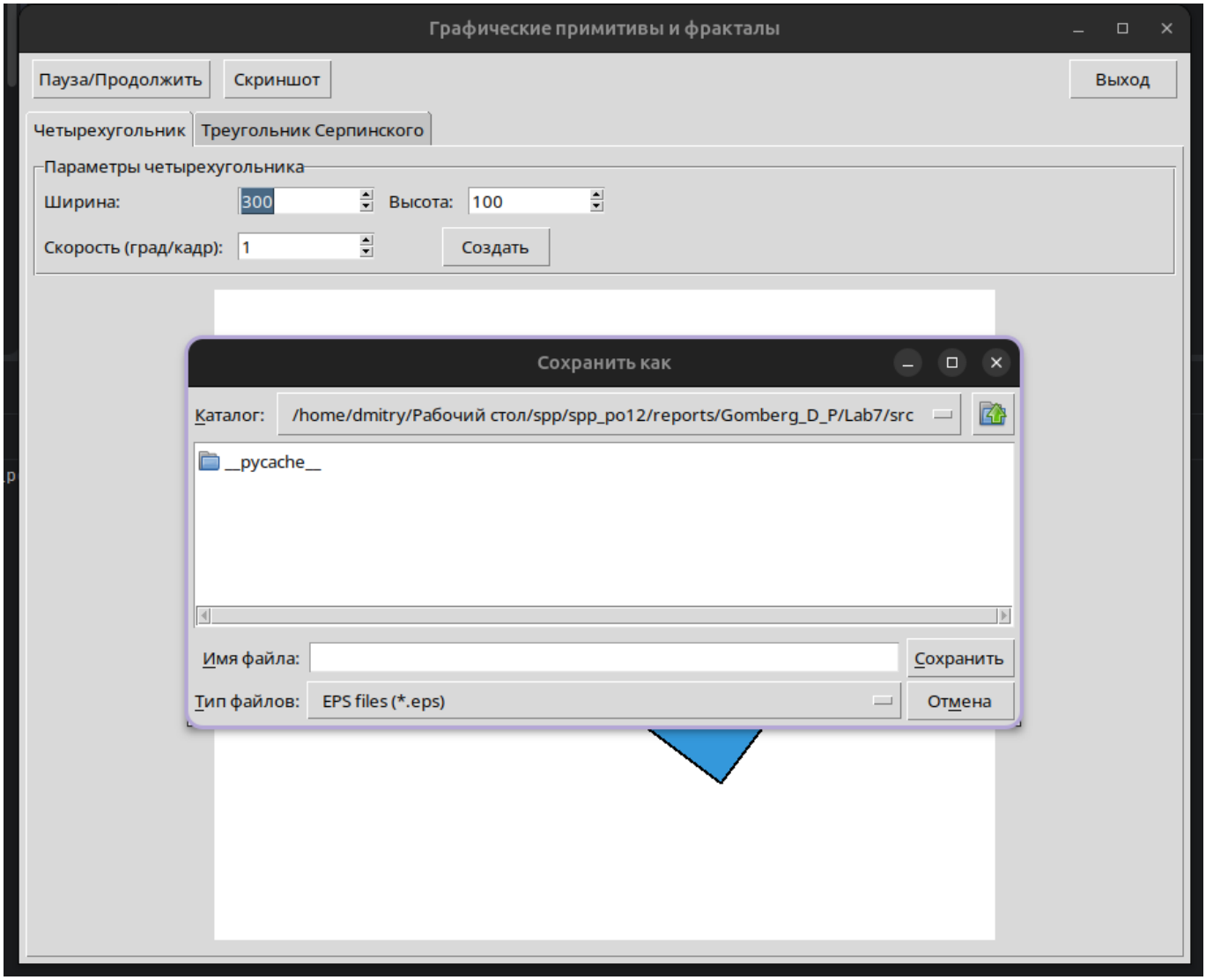
```
    def _draw_triangle(self, p1, p2, p3, depth):
        if depth == 0:
            color = self.color_map[0]
            points = list(p1) + list(p2) + list(p3)
            tid = self.canvas.create_polygon(points, fill=color, outline="black")
            self.triangle_ids.append(tid)
        else:
            m12 = ((p1[0] + p2[0]) / 2, (p1[1] + p2[1]) / 2)
            m23 = ((p2[0] + p3[0]) / 2, (p2[1] + p3[1]) / 2)
            m31 = ((p3[0] + p1[0]) / 2, (p3[1] + p1[1]) / 2)

            self._draw_triangle(p1, m12, m31, depth - 1)
            self._draw_triangle(m12, p2, m23, depth - 1)
            self._draw_triangle(m31, m23, p3, depth - 1)


class GraphicsApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Графические примитивы и фракталы")
        self.root.geometry("900x700")

        self.paused = False
        self.animation_running = False
        self.quad = None
        self.sierpinski = None

        self.setup_ui()

    def setup_ui(self):
        control_frame = ttk.Frame(self.root)
        control_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

        notebook = ttk.Notebook(self.root)
        notebook.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        self.tab1 = ttk.Frame(notebook)
        self.tab2 = ttk.Frame(notebook)
        notebook.add(self.tab1, text="Четырехугольник")
        notebook.add(self.tab2, text="Треугольник Серпинского")

        self.setup_quad_tab()
        self.setup_sierpinski_tab()
        self.setup_control_buttons(control_frame)

    def setup_quad_tab(self):
        input_frame = ttk.LabelFrame(self.tab1, text="Параметры четырехугольника")
        input_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)
```

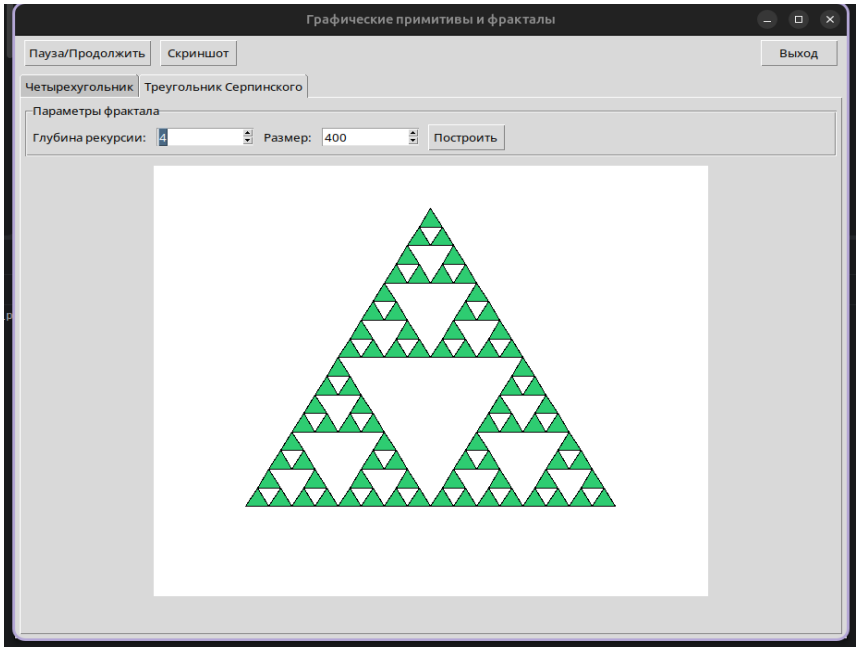
```
        ttk.Label(input_frame, text="Ширина:").grid(row=0, column=0, padx=5, pady=5, sticky=tk.W)
        self.quad_width = ttk.Spinbox(input_frame, from_=50, to=300, width=10)
        self.quad_width.set(150)
        self.quad_width.grid(row=0, column=1, padx=5, pady=5)

        ttk.Label(input_frame, text="Высота:").grid(row=0, column=2, padx=5, pady=5, sticky=tk.W)
        self.quad_height = ttk.Spinbox(input_frame, from_=50, to=300, width=10)
        self.quad_height.set(100)
        self.quad_height.grid(row=0, column=3, padx=5, pady=5)

        ttk.Label(input_frame, text="Скорость (град/кадр):").grid(row=1, column=0, padx=5, pady=5,
sticky=tk.W)
        self.quad_speed = ttk.Spinbox(input_frame, from_=1, to=20, width=10)
        self.quad_speed.set(3)
        self.quad_speed.grid(row=1, column=1, padx=5, pady=5)

        ttk.Button(input_frame, text="Создать", command=self.create_quad).grid(row=1, column=2,
columnspan=2, padx=5, pady=5)

        self.canvas1 = tk.Canvas(self.tab1, bg="white", width=600, height=500)
        self.canvas1.pack(padx=5, pady=5)

    def setup_sierpinski_tab(self):
        input_frame = ttk.LabelFrame(self.tab2, text="Параметры фрактала")
        input_frame.pack(side=tk.TOP, fill=tk.X, padx=5, pady=5)

        ttk.Label(input_frame, text="Глубина рекурсии:").grid(row=0, column=0, padx=5, pady=5,
sticky=tk.W)
        self.sierp_depth = ttk.Spinbox(input_frame, from_=1, to=7, width=10)
        self.sierp_depth.set(5)
        self.sierp_depth.grid(row=0, column=1, padx=5, pady=5)

        ttk.Label(input_frame, text="Размер:").grid(row=0, column=2, padx=5, pady=5, sticky=tk.W)
        self.sierp_size = ttk.Spinbox(input_frame, from_=200, to=500, width=10)
        self.sierp_size.set(400)
        self.sierp_size.grid(row=0, column=3, padx=5, pady=5)

        ttk.Button(input_frame, text="Построить", command=self.create_sierpinski).grid(row=0, column=4,
padx=5, pady=5)

        self.canvas2 = tk.Canvas(self.tab2, bg="white", width=600, height=500)
        self.canvas2.pack(padx=5, pady=5)

    def setup_control_buttons(self, frame):
        ttk.Button(frame, text="Пауза/Продолжить", command=self.toggle_pause).pack(side=tk.LEFT, padx=5)
        ttk.Button(frame, text="Скриншот", command=self.take_screenshot).pack(side=tk.LEFT, padx=5)
        ttk.Button(frame, text="Выход", command=self.root.quit).pack(side=tk.RIGHT, padx=5)

    def create_quad(self):
```

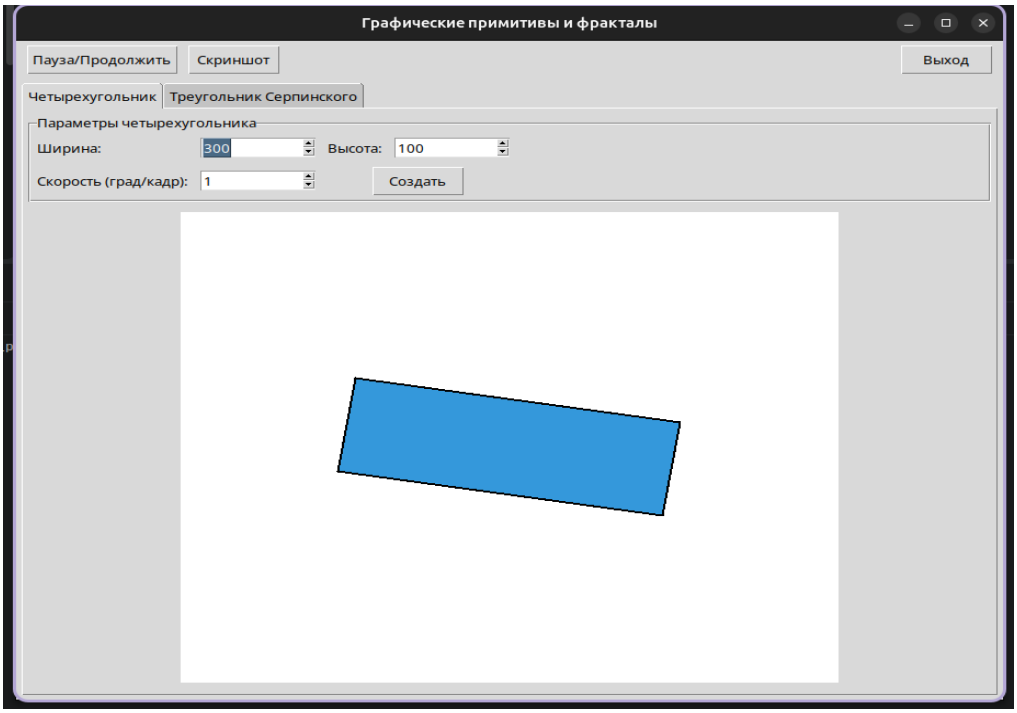
```
        width = int(self.quad_width.get())
        height = int(self.quad_height.get())

        self.canvas1.delete("all")
        self.quad = RotatingQuadrilateral(self.canvas1, 300, 250, width, height)
        self.quad.draw()

        if not self.animation_running:
            self.animation_running = True
            self.animate_quad()

    def animate_quad(self):
        if self.animation_running and self.quad:
            if not self.paused:
                speed = int(self.quad_speed.get())
                self.quad.update_angle(speed)
            self.root.after(30, self.animate_quad)

    def create_sierpinski(self):
        size = int(self.sierp_size.get())
        depth = int(self.sierp_depth.get())

        self.canvas2.delete("all")

        p1 = (300, 50)
        p2 = (300 - size / 2, 50 + size * math.sqrt(3) / 2)
        p3 = (300 + size / 2, 50 + size * math.sqrt(3) / 2)

        self.sierpinski = SierpinskiTriangle(self.canvas2, p1, p2, p3, depth)
        self.sierpinski.draw()

    def toggle_pause(self):
        self.paused = not self.paused

    def take_screenshot(self):
        try:
            notebook = self.root.nametowidget(self.root.winfo_children()[1])
            current_tab = notebook.index(notebook.select())

            if current_tab == 0:
                canvas = self.canvas1
            else:
                canvas = self.canvas2

            file_path = filedialog.asksaveasfilename(
                defaultextension=".eps",
                filetypes=[("EPS files", "*.eps"), ("All files", "*.*")]
            )
```

```
        if file_path:
            canvas.postscript(file=file_path, colormode="color",
                              width=canvas.winfo_width(), height=canvas.winfo_height())
            messagebox.showinfo("Скриншот", f"Сохранено: {file_path}")
    except Exception as e:
        messagebox.showerror("Ошибка", str(e))


def main():
    root = tk.Tk()
    app = GraphicsApp(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

Результат выполнения программы:







Вывод: освоили возможности языка программирования Python в разработке оконных приложений